

LAX LEAN ARCHIVE

Online Tutorial

Édouard Bonnet
CNRS, LIP (ENS Lyon)

Jan Dreier
HPI Potsdam

Clemens Kuske
HPI Potsdam

30 September 2026

What does it mean to formalize something in Lean?

Fermat's Last Theorem

For every integer $n \geq 3$ there are no positive integers a, b, c such that $a^n + b^n = c^n$.

What does it mean to formalize something in Lean?

Fermat's Last Theorem

For every integer $n \geq 3$ there are no positive integers a, b, c such that $a^n + b^n = c^n$.

```
theorem fermat_last_theorem :  
  ∀ n : ℕ, n ≥ 3 →  
    ¬ ∃ a b c : ℕ, a > 0 ∧ b > 0 ∧ c > 0 ∧  
      a ^ n + b ^ n = c ^ n := by
```



... 13 million more lines

What does it mean to formalize something in Lean?

Fermat's Last Theorem

For every integer $n \geq 3$ there are no positive integers a, b, c such that $a^n + b^n = c^n$.

```
theorem fermat_last_theorem :  
  ∀ n : ℕ, n ≥ 3 →  
    ¬ ∃ a b c : ℕ, a > 0 ∧ b > 0 ∧ c > 0 ∧  
      a ^ n + b ^ n = c ^ n := by
```

Lean's job

Check that the proof
is correct. No human
needs to read it.



... 13 million more lines

What does it mean to formalize something in Lean?

Our job

Check that the Lean statement says what the paper says. It is only a few lines.

Fermat's Last Theorem

For every integer $n \geq 3$ there are no positive integers a, b, c such that $a^n + b^n = c^n$.

```
theorem fermat_last_theorem :
```

```
  ∀ n : ℕ, n ≥ 3 →
```

```
    ¬ ∃ a b c : ℕ, a > 0 ∧ b > 0 ∧ c > 0 ∧  
      a ^ n + b ^ n = c ^ n := by
```

Lean's job

Check that the proof is correct. No human needs to read it.



... 13 million more lines

Statements are types, proofs are programs

Curry–Howard: checking a proof = checking that a program has a certain type.

Further reading: *Theorem Proving in Lean 4*. Chapters 1–4 are enough to read Lax statements.

Statements are types, proofs are programs

Curry–Howard: checking a proof = checking that a program has a certain type.

Natural language

STATEMENT

For every $n \in \mathbb{N}$ there is an m
with $n < m$.

Further reading: *Theorem Proving in Lean 4*. Chapters 1–4 are enough to read Lax statements.

Statements are types, proofs are programs

Curry–Howard: checking a proof = checking that a program has a certain type.

Natural language

STATEMENT

For every $n \in \mathbb{N}$ there is an m
with $n < m$.

PROOF

Proof. Fix $n \in \mathbb{N}$. Take $m =$
 $n + 1$. Then $n < m$.

Further reading: *Theorem Proving in Lean 4*. Chapters 1–4 are enough to read Lax statements.

Statements are types, proofs are programs

Curry–Howard: checking a proof = checking that a program has a certain type.

Natural language

STATEMENT

For every $n \in \mathbb{N}$ there is an m with $n < m$.

PROOF

Proof. Fix $n \in \mathbb{N}$. Take $m = n + 1$. Then $n < m$.

Dependent type theory

TYPE

$\forall n : \mathbb{N}, \exists m, n < m$

Functions that send each n to a proof that some m is larger.

Further reading: *Theorem Proving in Lean 4*. Chapters 1–4 are enough to read Lax statements.

Statements are types, proofs are programs

Curry–Howard: checking a proof = checking that a program has a certain type.

Natural language

STATEMENT

For every $n \in \mathbb{N}$ there is an m with $n < m$.

PROOF

Proof. Fix $n \in \mathbb{N}$. Take $m = n + 1$. Then $n < m$.

Dependent type theory

TYPE

$\forall n : \mathbb{N}, \exists m, n < m$

Functions that send each n to a proof that some m is larger.

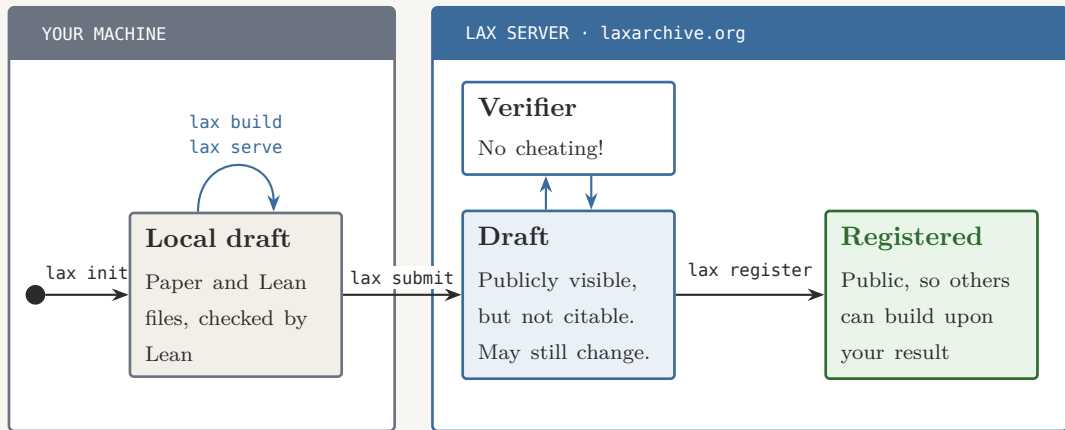
OBJECT OF THAT TYPE

`fun n => {  $n + 1$ , Nat.lt_add_one n }`
 witness m evidence $n < n + 1$

Further reading: *Theorem Proving in Lean 4*. Chapters 1–4 are enough to read Lax statements.

Live demo

How does a submission become part of the archive?



Concepts and Proofs

Concepts and Proofs

Concept file:

- the surface exposed on the website
- collection of declarations
forming a semantic unit
- ideally: simple, tactic-free Lean
- each statement (lemmas,
theorems, conjectures) is an `axiom`

Concepts and Proofs

Concept file:

- the surface exposed on the website
- collection of declarations forming a semantic unit
- ideally: simple, tactic-free Lean
- each statement (lemmas, theorems, conjectures) is an axiom

```
1 import Mathlib.Combinatorics.SimpleGraph.Acyclic
2 import Mathlib.Data.Nat.Lattice
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 namespace Lax228581.Treewidth
27
28
29
30
31
32 structure TreeDecomposition {V : Type} (G : SimpleGraph V) where
33   Node : Type
34   [nodeFintype : Fintype Node]
35   tree : SimpleGraph Node
36   isTree : tree.IsTree
37   bag : Node → Finset V
38   vertex_mem_bag : ∀ v : V, ∃ i : Node, v ∈ bag i
39   edge_mem_bag : ∀ {u v : V}, G.Adj u v → ∃ i : Node, u ∈ bag i ∧ v ∈ bag i
40   bag_indices_connected :
41     ∀ v : V, (tree.induce {i : Node | v ∈ bag i}).Connected
42
43
44
45
46
47
48
49
50
51
52
53
54 def HasTreewidthAtMost {V : Type} (G : SimpleGraph V) (w : ℕ) : Prop :=
55   ∃ D : TreeDecomposition G, ∀ i, (D.bag i).card ≤ w + 1
56
57
58
59 noncomputable def treewidth {V : Type} [Fintype V] [DecidableEq V]
60   (G : SimpleGraph V) : ℕ :=
61   sInf {w | HasTreewidthAtMost G w}
62
63 end Lax228581.Treewidth
```

Concepts and Proofs

Concept file:

- the surface exposed on the website
- collection of declarations forming a semantic unit
- ideally: simple, tactic-free Lean
- each statement (lemmas, theorems, conjectures) is an **axiom**

```
namespace Lax842588.MSOTreeAutomataEquivalence

open Lax842588.RankedTree
open Lax842588.TreeStructure
open Lax842588.TreeAutomaton

universe u v

/-- Every tree automaton with finitely many states has an equivalent MSO sentence. -/
axiom automaton_definable_by_mso {A : RankedAlphabet.{u}} {Q : Type v}
  [Fintype Q] (M : Automaton A Q) :
  ∃ phi : Lax146103.MSOSyntax.Sentence (treeSignature A), M.language = sentenceLanguage phi

/-- Every MSO sentence defines a recognizable ranked-tree language. -/
axiom mso_definable_is_recognizable {A : RankedAlphabet.{u}}
  (phi : Lax146103.MSOSyntax.Sentence (treeSignature A)) :
  Recognizable (sentenceLanguage phi)

/-- Recognizability and MSO definability characterize the same languages. -/
axiom recognizable_iff_msoDefinable {A : RankedAlphabet.{u}}
  (L : TreeLanguage A) :
  Recognizable L ↔
  ∃ phi : Lax146103.MSOSyntax.Sentence (treeSignature A), L = sentenceLanguage phi

end Lax842588.MSOTreeAutomataEquivalence
```

 [Show Proof](#) →

 [Show Proof](#) →

 [Show Proof](#) →

Concepts and Proofs

Concept file:

- the surface exposed on the website
- collection of declarations forming a semantic unit
- ideally: simple, tactic-free Lean
- each statement (lemmas, theorems, conjectures) is an `axiom`

```
1 import Lax434930.PolynomialTime
2 import Lax434930.NondeterministicPolynomialTime
3
4 /-!
5 ---
6 title: P versus NP
7 type: open question
8 ---
9 The P versus NP problem asks whether every language with polynomially
10 bounded, polynomial-time verifiable certificates can also be decided in
11 deterministic polynomial time. We state the conjectured separation
12  $P \neq NP$  as an open question.
13 -/
14
15 namespace Lax434930.PVersusNP
16
17 open PolynomialTime NondeterministicPolynomialTime
18
19 /-- The conjectured separation of P and NP, left as an open question. -/
20 axiom P_ne_NP : P ≠ NP
21
22 end Lax434930.PVersusNP
```

Concepts and **Proofs**

Proof file:

- for each axiom (to be proven) in each concept file,
there has to be a matching theorem in a proof file
with definitionally equal types
- need not be streamlined
this is what is lax about Lax
- remains in the submitter's public Git repo...
- ...but can also be searched and imported by an agent

Compositionality

Lax's guidelines encourage agents to search and reuse concepts

- no need to reformatize [insert painful foundation to formalize]

Compositionality

Lax's guidelines encourage agents to search and reuse concepts

- no need to reformatize [insert painful foundation to formalize]
- **concept** compositionality:
 - can import a definition, a proven or unproven axiom
 - the proof of a reused proven axiom is not type-checked again

Compositionality

Lax's guidelines encourage agents to search and reuse concepts

- no need to reformatize [insert painful foundation to formalize]
- **concept** compositionality:
 - can import a definition, a proven or unproven axiom
 - the proof of a reused proven axiom is not type-checked again
- **proof** compositionality:
 - import files from the proof folder
 - collected artifacts are used to type-check
 - decide if the reused block should become a concept

Demo – compositionality and faithfulness

Polynomial grid minor theorem of Chuzhoy–Tan

Theorem

Every graph with treewidth $\tilde{\Omega}(k^{10})$ admits a $k \times k$ grid minor.

130 pages across 10 papers

Polynomial grid minor theorem of Chuzhoy–Tan

Theorem

Every graph with treewidth $\tilde{\Omega}(k^{10})$ admits a $k \times k$ grid minor.

130 pages across 10 papers

Work period	Proof progress	Hours	Tokens (M)
May 14–June 1	0% $\rightarrow$ 30%	128.1	121.04
July 1–14	30% $\rightarrow$ 35%	28.6	35.41
July 15–21	35% $\rightarrow$ 55%	46.0	49.69
July 21–28	55% $\rightarrow$ 100%	41.5	46.89
July 28–29	Lax submission	13.0	4.83
Total	0% $\rightarrow$ 100%	257.1	257.86

Tokens = uncached input + output

Hours = agent-hours

Codex’s progress in formalizing in Lean

Lax submission	Hours	M tokens	n-l pages	pages/h
May 14–June 1 (5.5)	128.1	121	39	0.30
Poly grid minor	257.1	258	130	0.51
July 21–28 (5.6)	41.5	46.9	58.5	1.41
Arc Kayles hardness	6.8	2.59	12	1.77
Håstad’s inapprox	18.1	5.75	35	1.93

Codex weekly limit for Plus with 6-Astra high: 1.4M tokens (x5, x20 for Pro)

Codex’s progress in formalizing in Lean

Lax submission	Hours	M tokens	n-l pages	pages/M tokens
May 14–June 1 (5.5)	128.1	121	39	0.32
Poly grid minor	257.1	258	130	0.50
July 21–28 (5.6)	41.5	46.9	58.5	1.25
Arc Kayles hardness	6.8	2.59	12	4.63
Håstad’s inapprox	18.1	5.75	35	6.09

Codex weekly limit for Plus with 6-Astra high: 1.4M tokens (x5, x20 for Pro)

Why to use Lax?

- validating proof sketches, improving existing bounds, etc.
- checking paper drafts
- giving a convenient, possibly anonymized, alternative route for reviewers/readers to assess correctness
- building foundations to ease the above items in the future
- learning more Lean with a clear motivation

Related Projects

Autoformalization in Lean

Archives: Lean Pool, Palomar, Prove2Me/FormalPedia, TheoremForces, etc.

Libraries: merely true, Tau Ceti, etc.

Multi-agent scaffolds for autoformalization: AutoFormBot, RepoProver, LeanMarathon, Archon, LeanFlow, TeXRA, etc.

Start and current corpus

	Project	Launch	$\approx$ LOC (M)	Current bias
Archives	Lax	16 Sep 2026	1.2	TCS, graphs, algorithms
	Lean Pool	June 2026	4.24	
	Palomar	18 Aug 2026	2	maths \ TCS
	Prove2Me	28 Aug 2026	22.3	number theory, combinatorics
Libraries	Tau Ceti	July 2026	1.46	algebra, analysis, geometry
	 Mathlib	July 2017	2.33	maths \ TCS
	 CSLib	June 2025	0.04	TCS

 Maintained fully by hand (at least until mid-2026).

Concept reuse, validation and interfaces

Project	Concept reuse	CLI	Validation	Review	UI features
Lax	✓	✓	✓	human	A, S, PN, CD, SD, DV
Lean Pool	def reuse		✓	AI	A, S, PN, CD
Palomar			✓	AI	A, S
Prove2Me	✓		✓	human	A, S, PN

A: API/declaration browsing

S: search

PN: proof networks

CD: concept-dependency graphs

SD: submission-dependency graphs

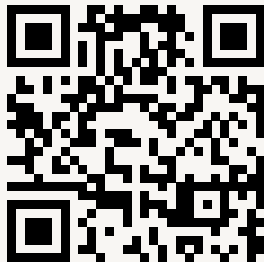
DV: Paper/Lean dual view

Live demo

Contributing to LAX

- Formalize and share your results.

Join the LAX Discord

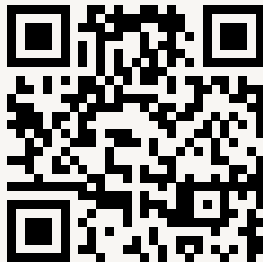


<https://discord.gg/Dqu3HTtch>

Contributing to LAX

- Formalize and share your results.
- Discuss pitfalls and successes.

Join the LAX Discord

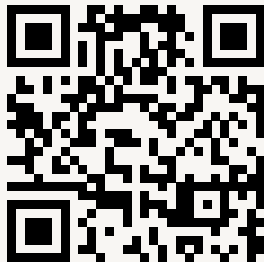


<https://discord.gg/Dqu3HTtch>

Contributing to LAX

- Formalize and share your results.
- Discuss pitfalls and successes.
- Endorse submissions.

Join the LAX Discord

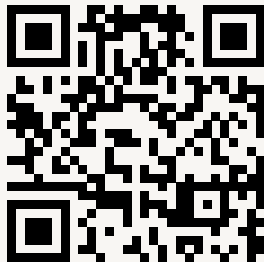


<https://discord.gg/Dqu3HTtch>

Contributing to LAX

- Formalize and share your results.
- Discuss pitfalls and successes.
- Endorse submissions.
- Report bugs and share suggestions for LAX.

Join the LAX Discord

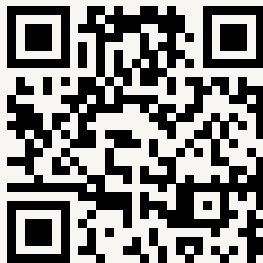


<https://discord.gg/Dqu3HTtch>

Contributing to LAX

- Formalize and share your results.
- Discuss pitfalls and successes.
- Endorse submissions.
- Report bugs and share suggestions for LAX.
- Work together on formalizing concepts.

Join the LAX Discord

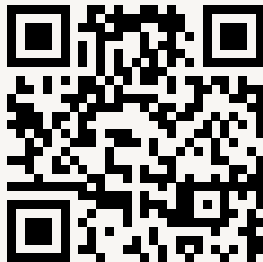


<https://discord.gg/Dqu3HTtch>

How to work together

- Ask colleagues to review your work.

Join the LAX Discord

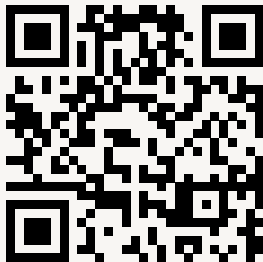


<https://discord.gg/Dqu3HTtch>

How to work together

- Ask colleagues to review your work.
- Work together on the basics or a larger result.

Join the LAX Discord

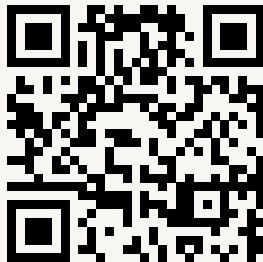


<https://discord.gg/Dqu3HTtch>

How to work together

- Ask colleagues to review your work.
- Work together on the basics or a larger result.
- Form taskforces around basics of your field.

Join the LAX Discord



<https://discord.gg/Dqu3HTtch>

Meeting for the Americas

This meeting is repeated for participants in US time zones and non-attendees.

Wednesday, 7 October 2026

19:00 CEST

Questions and remarks

What would you like to discuss?

How does a submission become part of the archive?

